Projet : identifier les oiseaux d'une mangeoire

RobIA LP2 - R3.14 Deep Learning

---

1. Contexte

Un naturaliste a installé un piège photo devant une mangeoire de jardin, déclenché au mouvement. La caméra a produit plusieurs milliers d'images sur plusieurs semaines, et personne ne va les regarder une par une. Son besoin : **savoir quelles espèces sont passées**.

Il vous faut donc un système qui prend une image brute sortie de la caméra et identifie l'espèce d'oiseau présente. Le problème n'est pas résolu d'avance : les images du piège photo ne sont pas annotées, et le seul jeu étiqueté dont vous disposez vient d'une autre source, avec d'autres conditions de prise de vue. Le cœur du projet est donc de faire fonctionner un modèle entraîné sur un dataset pour prédire sur un autre.

---

2. Ce qui est noté

Note commune au binôme : **rendu 70%, soutenance 30%**.

Le rendu est jugé sur la démarche expérimentale, la rigueur de votre méthodologie au cours des expériences, l'analyse de vos résultats et des erreurs, et la qualité du code. La performance brute ne vaut qu'un bonus : un score élevé obtenu par une fuite de données (le modèle apprend sur le test) sera pénalisé.

La soutenance doit porter sur **vos** décisions et leur justification : expliquez ce que vous avez choisi, pourquoi, ce que ça a donné, l'analyse des erreurs, etc. Un projet qui atteint 45% d'accuracy en expliquant précisément pourquoi vaut mieux qu'un projet à 70% dont personne ne sait d'où vient le chiffre.

Comment j'obtiens votre score. Je garde de côté des journées entières du piège photo qui ne vous sont pas distribuées, et je lance votre commande d'inférence dessus en suivant votre README. Si votre dépôt ne tourne pas chez moi, ou met un temps déraisonnable, je vous envoie ces images et vous me renvoyez le CSV : vous ne perdez donc pas votre note de performance, mais la **reproductibilité** de votre dépôt, elle, est notée et en pâtit. C'est le seul endroit où un README bâclé vous coûte des points.

---

3. Votre périmètre : le `classes.json`

Avant d'entraîner quoi que ce soit, vous devez décider quels **labels** votre modèle sait produire. Cette liste est votre périmètre, et c'est une décision notée : elle définit ce que votre modèle sait faire et ce qu'il ne saura jamais.

Elle se déclare dans un fichier `classes.json` comme ceci :

```
{"classes": ["Streptopelia_decaocto", "Erithacus_rubecula", "Parus_major",
"autre_oiseau", "rien"]}
```

Les règles de ce fichier :

- **Les noms d'espèces sont ceux de l'annexe B**, en latin et à l'orthographe exacte. C'est ce qui permet de confronter votre sortie à mes annotations.
- `incertain` **est interdit.** C'est une étiquette d'annotation, utile quand vous hésitez devant une image, pas une sortie de modèle.
- **La classe** `rien`**.** Une proportion importante des images du piège photo ne contient aucun oiseau. Un modèle qui ne sait pas dire `rien` doit nommer une espèce sur une mangeoire vide, et se trompe donc à chaque fois. De même, `autre_oiseau` vous permet de répondre juste sur une espèce que vous n'avez pas déclarée.

Comment ce choix est noté. Réduire votre périmètre ne vous pénalise pas mécaniquement : pour évaluer, je projette mes annotations dans votre vocabulaire, et toute espèce que vous n'avez pas déclarée devient `autre_oiseau`. Mais l'accuracy de votre modèle sera calculée en fonction du nombre de classes déclarées, et plus votre nombre de classes est petit, plus sa justification doit être solide.

---

4. Ce que vous rendez

1. **Un rapport de 3-6 pages** : résultat chiffré de votre exploration, choix de périmètre justifié, protocole d'évaluation, un tableau des gains (une ligne par étape : ce que vous avez changé, le score obtenu, et ce que cette étape a apporté par rapport à la précédente), analyse d'erreurs, limites de votre système, etc.
2. **Un dépôt de code reproductible** : arborescence claire, `requirements.txt`, et un **README** donnant les deux commandes de l'annexe A — une pour l'entraînement, une pour l'inférence.
3. **Votre modèle final** : les poids dans le dépôt, avec le `classes.json` correspondant. Votre script d'inférence doit savoir où les trouver tout seul.
4. **Une soutenance de 10 minutes**, suivie de questions.

---

5. Règles impératives

Critères sur lesquels vous serez notés, pensez bien à les relire plusieurs fois avant de vous lancer dans des expériences.

R1. Tout résultat annoncé doit être mesuré sur des données que le modèle n'a jamais vues. Y compris chaque ligne de votre tableau des gains.

R2. Découpez vos données par jour, jamais par fichier. Toutes les images d'une même journée vont entièrement dans l'entraînement, ou entièrement dans la validation. Jamais les deux.

La raison : deux images prises au même instant, ou à quelques secondes d'intervalle lors du même passage d'oiseau, sont quasiment identiques. Si l'une sert à entraîner et l'autre à valider, votre modèle est noté sur une image qu'il a déjà vue.

R3. Toute technique ajoutée doit être accompagnée de son gain mesuré. Une amélioration que vous ne savez pas chiffrer ne compte pas, même si le score final est bon.

R4. Votre dépôt doit pouvoir être lancé par quelqu'un d'autre que vous. Vous choisissez librement vos fichiers, vos noms de scripts et vos arguments : je n'impose aucune interface. En échange, votre README donne les deux commandes de l'annexe A, chacune prenant le chemin des données en paramètre et écrivant

dans un dossier de sortie — **aucun chemin absolu dans votre code**, les données ne seront pas au même endroit chez moi. C'est votre commande d'inférence que je lance sur des journées que vous n'avez jamais vues, et son CSV de sortie qui produit votre note de performance.

---

6. Donnée personnelle

Des personnes apparaissent sur certaines images du piège photo. La règle : les images doivent rester sur votre machine et ne pas en sortir, pas de publication en ligne.

- Rapport et soutenance : les images sont autorisées, personnes incluses, à condition que ces documents ne soient transmis qu'à moi. Pas de portfolio ou GitHub public.
- **Créez votre `.gitignore` avant votre premier commit**, avec au minimum les lignes suivantes. Un dépôt où les images sont déjà parties est un dépôt qu'on ne rattrape pas :

```
Data_Hackaton/
Oiseaux_Laurens/
*.pt
```

---

7. Organisation pratique

Le jeu de données est petit et les modèles demandés sont légers : l'essentiel de vos expériences tourne sur votre machine ou sur un GPU gratuit en ligne. Travaillez en basse résolution et avec une limite d'images en entrée pour entraîner vite, et augmentez pour les entraînements GPU.

Runs sur le GPU de l'IUT. Pour les entraînements lourds et votre modèle final, on va essayer de mettre en place des runs sur le GPU de l'IUT pour permettre des entraînements plus longs et plus complexes.

---

8. Les données

Durant ce projet, vous aurez accès à deux jeux de données de nature très différente :

`Data_Hackaton` : photos déjà classifiées, triées par espèce (un dossier par espèce, nom latin). C'est votre seule source d'images labellisées au départ. Beaucoup d'espèces sont présentes, dont la plupart n'ont aucune chance d'apparaître devant une mangeoire de jardin. Les conditions de prise de vue sont très variées : l'oiseau peut remplir le cadre ou n'occuper que quelques pixels au fond d'un buisson.

`Oiseaux_Laurens` : les images brutes du piège photo, non annotées, réparties en plusieurs dossiers (les différents dumps de la carte mémoire). C'est le domaine cible, celui sur lequel votre système sera jugé. Les métadonnées EXIF de chaque fichier contiennent la **date et l'heure** de prise de vue : cette information vous sera indispensable.

L'annotation fait partie du travail

La difficulté du projet réside dans le fait que `Oiseaux_Laurens/` n'est pas annoté. Pour évaluer quoi que ce soit sur ce dataset, il vous faut donc étiqueter vous-mêmes une partie des photos, avec la grille de l'annexe B. Combien ? À vous de voir : les plus déterminés en feront beaucoup, et en tireront une évaluation d'autant plus

solide. Vous pouvez aussi vous mettre en commun entre binômes, rien ne vous y oblige, mais annoter à dix couvre dix fois plus de séquences.

Disclaimer : hors performances inouies, même à la fin du projet personne ne saura avec certitude quelles espèces sont passées devant cette mangeoire. Votre résultat final sur `Oiseaux_Laurens/` n'est donc pas une accuracy au dixième près, c'est une comparaison entre ce que votre modèle prédit et ce que vous avez observé et annoté. C'est cette confrontation que je note, pas le chiffre.

Mais rappellez vous que `Data_Hackaton/` est déjà étiqueté. Vous devez alors réserver une part que vous n'entraînez jamais pour évaluer vos premiers modèles.

/!\ L'analyse de données est une étape cruciale du projet

Vous ne pouvez pas traiter ces images comme une liste de fichiers indépendants. Avant toute chose, explorez :

- **Comptez ce que vous chargez.** Le nombre de fichiers dans un dossier n'est pas le nombre d'images exploitables. Vérifiez que ce que vous donnez à votre `Dataset` est bien ce que vous croyez.
- **Cherchez la redondance.** Une caméra à déclenchement ne produit pas une image isolée par événement. Plusieurs fichiers peuvent décrire exactement le même instant, et un même passage d'oiseau produit une rafale d'images quasi identiques. Regardez de près les noms de fichiers et les résolutions, mesurez ce que vous trouvez, et décidez quoi en faire.
- **Regardez les temps de lecture.** Toutes les images de ce jeu ne mettent pas le même temps à ouvrir. Cet écart peut être significatif et pénaliser vos temps d'entrainement.
- **Comparez les deux jeux.** Qu'est-ce qui change entre une photo naturaliste et une image de piège photo ? Listez les différences, elles définissent tous vos choix ensuite.

Votre rapport doit contenir le résultat chiffré de cette exploration.

---

9. Les étapes

Les étapes se suivent dans cet ordre : chacune réutilise la chaîne construite à la précédente. Il n'y a pas de calendrier imposé, organisez-vous comme vous voulez jusqu'au 13 novembre.

étape 0 : explorer et préparer

Analysez les deux jeux (section 8), construisez votre dataset de travail, décidez de votre découpage selon R2. Vous devez également décider de **quelles espèces vous cherchez à reconnaître** : ce choix vous appartient mais devra être justifié. Toutes les espèces disponibles ne sont pas pertinentes, et les espèces ne sont pas représentées de manière équitable.

étape 1 : une baseline honnête

Un CNN entraîné de zéro, sur les images entières, à partir du jeu étiqueté. Évaluez-le deux fois : sur une part de `Data_Hackaton` que vous n'avez jamais entraînée, puis sur les séquences du piège photo que vous avez annotées. L'écart entre les deux est votre première mesure de l'écart de domaine.

Le résultat sur le piège photo sera probablement mauvais : c'est attendu, et c'est le point de référence auquel vous comparerez tout le reste.

étape 2 : transfer learning

Pour l'instant votre baseline part de zéro : elle doit apprendre à reconnaître des contours, des textures et des plumes avec quelques centaines d'images. Pour éviter ça, on peut utiliser un réseau pré-entraîné sur ImageNet qui a déjà appris tout ça sur 1,2 million d'images. Vous n'avez plus qu'à lui apprendre vos classes.

En pratique, `torchvision.models` fournit les architectures et leurs poids pré-entraînés. Un ResNet18 est un bon point de départ :

```
from torchvision import models
modele = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
modele.fc = torch.nn.Linear(modele.fc.in_features, len(classes))
```

La dernière couche d'origine sort 1 000 scores, un par classe ImageNet. Vous la remplacez par la vôtre, qui sort autant de scores que d'espèces dans votre `classes.json`. Le reste du réseau est conservé tel quel.

Les points sur lesquels on se plante systématiquement la première fois :

- Les images d'entrée doivent ressembler à celles d'ImageNet, sinon les couches pré-entraînées reçoivent n'importe quoi. Redimensionnez à la résolution attendue par l'architecture (224×224 pour un ResNet18) et normalisez avec les moyennes et écarts-types d'ImageNet, que `models.ResNet18_Weights.DEFAULT.transforms()` vous donne directement.
- L'augmentation s'applique à l'entraînement seulement, jamais à la validation. Sinon vous ne mesurez plus rien de stable.

Mesurez le gain par rapport à l'étape 1, avec le même protocole d'évaluation.

étape 3 : détecter avant de classer

Jusqu'ici vous donnez l'image entière à votre classifieur, et ça lui pose deux problèmes.

Il apprend le décor en même temps que l'oiseau, car il ne sait pas quelle partie de l'image vous intéresse, et il gaspille sa résolution (l'oiseau n'occupe souvent qu'une petite part du cadre).

Notre solution va être de détecter l'oiseau d'abord, le classer ensuite. Un détecteur d'objets ne dit pas quelle espèce est présente, il dit où il y a un oiseau, et vous obtenez une image qui ne contient presque que l'oiseau en sortie.

Vous n'entraînez pas ce détecteur : vous en prenez un déjà entraîné. La bibliothèque **`ultralytics`** (`pip install ultralytics`) permet d'utiliser les modèles **YOLO**, entraînés sur un jeu de 80 classes d'objets courants qui comporte une classe `bird`.

Voici un exemple de code pour détecter les oiseaux dans une image et récupérer leurs coordonnées (pour aller plus loin, allez voir la doc !):

```
from ultralytics import YOLO
detecteur = YOLO("yolo11s.pt")
resultats = detecteur(chemin_image, classes=[14])  # 14 = "bird" dans COCO
for boite in resultats[0].boxes:
```

```
    x1, y1, x2, y2 = boite.xyxy[0].tolist()
    confiance = float(boite.conf[0])
```

Appliquez cette passe aux deux jeux : c'est tout l'intérêt, vos images d'entraînement et vos images de test doivent subir le même traitement, puis réentraînez votre classifieur sur les petites images.

Trois points à traiter explicitement :

- Réglez le seuil de confiance du détecteur sur l'accuracy finale de classification, pas sur le taux de détection. Un seuil bas détecte plus, mais recadre aussi des buissons, et une image fausse est pire qu'une image entière.
- Quand le détecteur ne trouve rien, décidez quoi faire : jeter l'image vous coûte des données d'entraînement.
- Sur le piège photo, une absence de détection est une information en soi. Exploitez-la pour la classe "pas d'oiseau" au lieu de la traiter comme un échec.

étape 4 : d'où doivent venir les données d'entraînement ?

La question centrale du projet. Entraînez votre modèle dans trois configurations :

1. uniquement sur le jeu étiqueté d'origine,
2. uniquement sur les images du piège photo que vous avez annotées,
3. sur les deux combinés.

Qu'apporte chaque source, et pourquoi ? C'est pour cette étape qu'il faut annoter, car sans étiquettes de mangeoire les configurations 2 et 3 n'existent pas et vous perdez l'étape centrale du projet.

étape 5 : décider sur une visite, pas sur une image

On ne cherche pas à étiqueter une image isolée mais à savoir quelle espèce est passée. Regroupez les images d'un même passage et agrégez les prédictions (vote majoritaire, moyenne des probabilités, ou mieux si vous trouvez).

Ce regroupement est ce que vous déclarez dans la colonne `sequence_id` du CSV (annexe A), et je le compare au mien. Mesurez l'écart entre votre score au niveau image et au niveau passage, et expliquez-le.

---

10. Pistes d'approfondissement

À traiter si vous avez consolidé les étapes précédentes. Choisissez selon ce qui vous intéresse, une piste bien traitée vaut mieux que trois survolées.

- Que le modèle sache dire "je ne sais pas" (le moins possible, mais quand c'est necessaire).
- Regarder ce que regarde le modèle. Vous pouvez utiliser Grad-CAM ou équivalent pour vérifier qu'il s'appuie sur l'oiseau et non sur le décor. Couplez-le à une analyse d'erreurs : quelles espèces se confondent, et pourquoi.
- Les images de nuit. Les prises de vue infrarouges en noir et blanc sont un régime absent du jeu d'entraînement d'origine.

Annexe A : ce que je dois pouvoir lancer

Je n'impose aucune architecture : vos noms de fichiers, vos arguments, votre structure. En échange, votre README donne deux commandes, dans une section "Entraînement" et une section "Inférence", que je lance depuis la racine de votre dépôt, exemple :

```
train.py <racine des données> <dossier de sortie>

predict.py <dossier d'images> <dossier de sortie>
```

Attention, il ne faut aucune interaction : ni `input()`, ni notebook à dérouler cellule par cellule.

Le CSV de prédictions

En-tête exact, une ligne par fichier d'entrée, aucune image omise :

```
image_id,sequence_id,label,confidence
```

- `image_id` : le nom du fichier, sans le chemin ni l'extension. C'est la clé qui me permet de comparer vos prédictions à celles des autres binômes.
- `sequence_id` : le passage auquel votre chaîne rattache l'image (étape 5). Forme libre, mais deux images d'un même passage portent le même identifiant.
- `label` : un des labels de votre `classes.json`, et rien d'autre.
- `confidence` : entre 0 et 1.

Annexe B : labels d'annotation

Grille à utiliser pour annoter les images du piège photo. Tenez-vous-y même en travaillant seuls : c'est ce qui permet de fusionner vos annotations avec celles des autres, et c'est le vocabulaire dans lequel votre `classes.json` sera lu.

On vous conseille d'annoter une séquence entière d'un coup plutôt que chaque image séparément (une séquence est la rafale d'images déclenchées lors d'une même visite d'oiseau, que vous reconstituez à partir des dates EXIF). Une seule étiquette vaut alors pour toutes ses images, et vous passez de plusieurs milliers d'images à quelques centaines de passages.

Nom latin	Nom français
Streptopelia_decaocto	Tourterelle turque
Erithacus_rubecula	Rouge-gorge familier
Fringilla_coelebs	Pinson des arbres
Parus_major	Mésange charbonnière
Turdus_merula	Merle noir

Nom latin	Nom français
Cyanistes_caeruleus	Mésange bleue
Pica_pica	Pie bavarde
Passer_domesticus	Moineau domestique
Sturnus_vulgaris	Étourneau sansonnet
Chloris_chloris	Verdier d'Europe
Carduelis_carduelis	Chardonneret élégant
Upupa_epops	Huppe fasciée
Garrulus_glandarius	Geai des chênes
Serinus_serinus	Serin cini
Sylvia_atricapilla	Fauvette à tête noire
Columba_palumbus	Pigeon ramier

Trois étiquettes supplémentaires :

- `autre_oiseau` : un oiseau, mais aucune espèce de la liste.
- `rien` : aucun oiseau (image vide, mammifère, personne, nuit noire).
- `incertain` : un oiseau est présent mais vous n'arrivez pas à trancher. À utiliser sans hésiter, une annotation inventée vaut moins que pas d'annotation.